PasteRobot — ПЛИС-часть робота нанесения пасты

Цепочка от ПК до драйверов моторов: строка G-кода из Termite → импульсы PUL/DIR на TB6600 → ответ `ok` / `err N`. Всё на SystemVerilog, сброс `rst` активен единицей, автоматы — в два блока (`always_comb` + `always_ff`), как в вашем `uart_tx.sv`.

Состояние на 08.10.2026: **все 9 тестов проходят в Questa 2025.2**, проект собирается в Quartus 25.1 для EP4CE6E22C8 (см. раздел 6).

---

1. Папки и файлы

У каждого модуля своя папка и три файла: **модуль**, **тест** (`tb_…sv`), **запуск теста** (`run_tb_…do`). Номер папки — место модуля на пути данных.

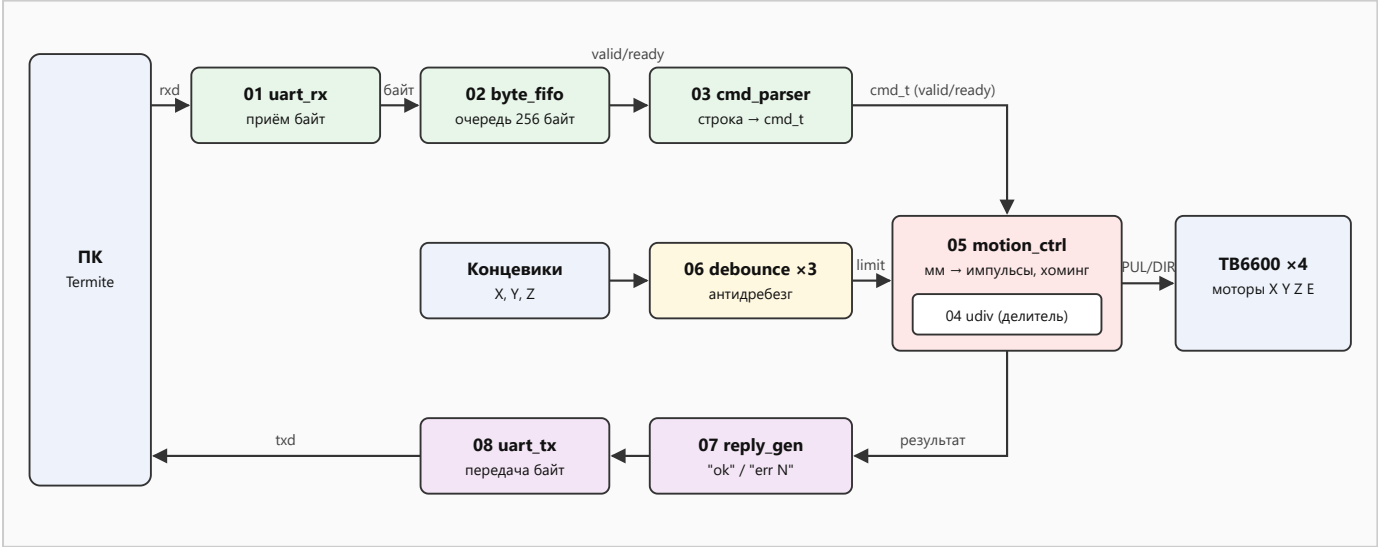
```
C:\FPGA_PROJECT\PasteRobot\
│  README.md              ← этот файл
│  run_all.bat            ← все тесты одной командой
│
├─ common\robot_pkg.sv    ← общий "словарь": оси, коды команд, коды ошибок, структура cmd_t
│
├─ 01_uart_rx\            приём байт с ПК              uart_rx.sv     tb_uart_rx.sv     run_tb_uart_rx.do
├─ 02_byte_fifo\          очередь байт                 byte_fifo.sv   tb_byte_fifo.sv   run_tb_byte_fifo.do
├─ 03_cmd_parser\         разбор строки G-кода         cmd_parser.sv  tb_cmd_parser.sv  run_tb_cmd_parser.do
├─ 04_udiv\               делитель (мм → импульсы)     udiv.sv        tb_udiv.sv        run_tb_udiv.do
├─ 05_motion_ctrl\        импульсы на моторы, хоминг   motion_ctrl.sv tb_motion_ctrl.sv run_tb_motion_ctrl.do
├─ 06_debounce\           антидребезг концевиков       debounce.sv    tb_debounce.sv    run_tb_debounce.do
├─ 07_reply_gen\          ответ "ok" / "err N"         reply_gen.sv   tb_reply_gen.sv   run_tb_reply_gen.do
├─ 08_uart_tx\            передатчик (ваш, из Uart\TX) uart_tx.sv     tb_uart_tx.sv     run_tb_uart_tx.do
└─ 09_robot_top\          всё вместе, для платы        robot_top.sv   tb_robot_top.sv   run_tb_robot_top.do
```

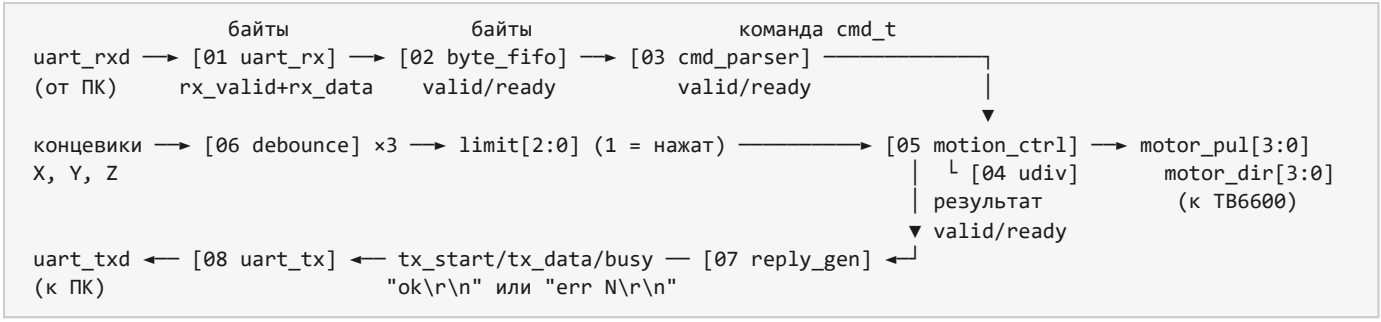
Ваши папки `C:\FPGA_PROJECT\Uart\RX` и `Uart\TX` не тронуты. В `08_uart_tx` лежит **копия** вашего передатчика и теста; в копию теста добавлена одна строка `errors++;` в «стороже», чтобы таймаут считался ошибкой.

После запусков в папках появляются `work\`, `transcript`, `sim.log` — это результаты симуляции, их можно удалять.

---

2. Как модули связаны





Все соединения, кроме выхода `uart_rx`, сделаны **рукопожатием valid/ready**:

- `valid = 1` — «у меня есть данные, они на шине»;
- `ready = 1` — «я готов их взять»;
- передача происходит на том фронте `clk`, где **оба** равны 1. Пока `ready = 0`, отправитель держит данные.

Благодаря этому модули ждут друг друга сами: пока моторы едут, `motion_ctrl` держит `cmd_ready = 0`, парсер не берёт новые байты (`in_ready = 0`), и следующая строка спокойно лежит в очереди `byte_fifo`.

Путь одной команды `G1 X20 Y8 E2` (Enter в Termite)

Шаг	Модуль	Что происходит	Куда уходит
1	`uart_rx`	Принимает 14 байт: `G1 X20 Y8 E2` и CR LF. На каждый байт — импульс `rx_valid`	`byte_fifo`
2	`byte_fifo`	Складывает байты в очередь (256 байт)	`cmd_parser`, по `valid/ready`
3	`cmd_parser`	Разбирает строку по символу. На конце строки выдаёт `cmd_t`: `OP_G1`, X = 2000, Y = 800, E = 200 (сотые)	`motion_ctrl`
4	`motion_ctrl` + `udiv`	Переводит мм в импульсы: `(2000 × k + 5000) / 10000`. Считает, сколько ехать от текущей точки, выставляет DIR	свои регистры
5	`motion_ctrl`	Выдаёт импульсы PUL на X, Y, E по Брезенхему: оси заканчивают одновременно, паста ложится равномерно	выводы → TB6600
6	`motion_ctrl`	Последний импульс отдан — результат `ERR_NONE`	`reply_gen`
7	`reply_gen` + `uart_tx`	Шлёт `ok` + CR LF	Termite на ПК

---

3. Модули

Подробное описание, временные диаграммы и схема автомата — в шапке каждого `.sv` файла. Здесь — коротко.

common/robot_pkg.sv — общий словарь

Номера осей (`AX_X = 0, AX_Y = 1, AX_Z = 2, AX_E = 3`), коды команд (`OP_G0, OP_G1, OP_G28, OP_M92`), коды ответа (`ERR_NONE … ERR_HOMING`) и структура `cmd_t` — разобранная строка. Компилируется первым.

01 uart_rx — приём байт

Синхронизатор из двух триггеров → автомат `IDLE → START → DATA → STOP` (+ `WAIT_IDLE` после ошибки). Читает линию в середине каждого бита. Отсеивает помехи короче половины бита. Неправильный стоп-бит →

`frame_err`, байт не выдаётся. **Выход:** `rx_data` + импульс `rx_valid` → `byte_fifo`.

02 byte_fifo — очередь байт

Память на 256 байт (в Quartus — блок M9K) и два указателя. Выходной байт выставлен заранее, без лишнего такта. Если очередь полна, новый байт теряется и загорается `overflow` (до сброса). **Вход:** от `uart_rx`. **Выход:** `rd_valid / rd_data / rd_ready` → `cmd_parser`.

03 cmd_parser — разбор строки

Автомат на 10 состояний читает строку **по одному символу** и не хранит её целиком. Понимает: `G0`, `G1`, `G28`, `M92`; числа со знаком, запятой или точкой (до 2 знаков после); маленькие буквы; поля без пробелов (`G1X5Y3`); комментарии после `;`; концы строки CR, LF, CR LF. Пустые строки пропускает без ответа. Проверяет правила из «путь-команды» (G0 без E, Z только отдельно и т. д.). **Выход:** `cmd_valid / cmd / cmd_ready` → `motion_ctrl`. Ошибка тоже уходит туда, внутри `cmd.err`, чтобы ответы на ПК шли строго по порядку.

04 udiv — делитель

Деление «столбиком», один бит частного за такт (50 тактов = 1 мкс). Нужен, чтобы перевести мм в импульсы без огромной комбинационной схемы, которая не успела бы за 50 МГц. **Используется внутри** `motion_ctrl`.

05 motion_ctrl — движение

Главный модуль логики. Выполняет команду:

- **M92** — запоминает коэффициенты `k` (импульсов на мм / на мкл), без перепрошивки;
- **G0 / G1** — для каждой оси `импульсов = (|значение| × k + 5000) / 10000`. X, Y, Z — абсолютные (едем от текущей точки к цели), E — сколько выдавить сейчас (`-` — втянуть пасту). Затем импульсы по Брезенхему: самая длинная ось шагает каждый период, остальные равномерно реже;
- **G28** — по очереди Z, X, Y едут к концевикам медленнее (`HOME_STEP_HZ`); где концевик нажат — там 0.

Безопасность: шаг **в сторону** нажатого концевика не делается → `err 5`. Концевик не найден за `HOME_MAX_STEPS` шагов → `err 6`. **Выход:** `step_pul / step_dir` → выводы на драйверы; результат `res_valid / res_err` → `reply_gen`.

Временные параметры импульсов (по умолчанию из «путь-команды»): импульс 10 мкс, пауза после смены DIR 10 мкс, частота 2 кГц.

06 debounce — концевики

Синхронизатор + фильтр: выход меняется, только если вход держит новое значение 2 мс подряд. **Выход:** чистый сигнал → `robot_top` → `limit[...]` для `motion_ctrl`.

07 reply_gen — ответ

Автомат `IDLE → SEND → WAIT_BUSY → WAIT_DONE` отдаёт передатчику по байту строку `ok\r\n` или `err N\r\n`. **Вход:** результат от `motion_ctrl`. **Выход:** `tx_start / tx_data` → `uart_tx`.

08 uart_tx — передатчик

Ваш модуль из `Uart\TX` без изменений.

09 robot_top — всё вместе

Только соединяет модули проводами, делает сброс из кнопки (`key_rst_n`) и подстраивает уровни под плату параметрами `DIR_INVERT`, `LIMIT_ACTIVE_LOW`, `ENA_LEVEL`, `LED_ON_LEVEL`. Светодиоды: `led_busy`, `led_error`,

`led_overflow`.

---

4. Команды и ответы

Команда	Что делает	Пример
`G28`	хоминг всех осей (Z, потом X, потом Y)	`G28`, или только `G28 X`
`G0 X.. Y..`	переезд без пасты	`G0 X10 Y5`
`G0 Z..`	поднять / опустить шприц (только отдельно)	`G0 Z2`
`G1 X.. Y.. E..`	дорожка: едем и выдавливаем E мкл за время пути	`G1 X20 Y8 E2`
`G1 E..`	точка: стоим и выдавливаем	`G1 E0,5`
`M92 X.. Y.. Z.. E..`	коэффициенты k: импульсов на мм (X, Y, Z) и на мкл (E)	`M92 X400 Y400 Z400 E92,2`

После сброса k = 400 имп/мм по осям и 92,2 имп/мкл (пример из «путь-команды»).

Ответ	Причина
`ok`	команда выполнена
`err 1`	неизвестная команда (не G0/G1/G28/M92)
`err 2`	лишний символ, буква не X/Y/Z/E, поле повторено
`err 3`	неверное число: нет цифр, больше 2 знаков после запятой, больше 99999,99, слишком много импульсов
`err 4`	недопустимое сочетание: G0 с E, Z вместе с X/Y, G1 с Z, G28 с E, M92 с k ≤ 0
`err 5`	шаг в сторону нажатого концевика
`err 6`	при хоминге концевик не нажался

---

5. Как запускать тесты

Перед запуском закройте окно Questa: бесплатная лицензия даёт только одну сессию. **Используйте `-batch`, а не `-c`:** с `-c` у вас симуляция зависает на `RPC_Init` (мешает антивирус 360).

Все тесты сразу

```
cd C:\FPGA_PROJECT\PasteRobot
```

```
.\run_all.bat
```

Ожидаемый результат — девять строк `PASSED` и `ALL TESTS PASSED` (около минуты). Полный вывод каждого теста — в `<папка>\sim.log`.

Один тест в консоли

```
cd C:\FPGA_PROJECT\PasteRobot\05_motion_ctrl
```

```
vsim -batch -do run_tb_motion_ctrl.do
```

Один тест в окне Questa (с осциллограммами)

1. `File → Change Directory…` → папка модуля, например `05_motion_ctrl`.
2. В строке Transcript: `do run_tb_motion_ctrl.do`
3. Откроется Wave с уже добавленными сигналами (входы, состояние автомата, выходы). Симуляция останется загруженной.

Как понять, что тест прошёл

1. После `vlog`: `Errors: 0`. Если ошибка компиляции — скрипт сразу выходит (`onerror`), старая сборка из `work` не запустится.
2. Строки `OK …` или `step N: … - done` по ходу теста, ни одной строки `ERROR`.
3. В конце `TEST PASSED`, а `$finish` — из основной части теста, а не «сторожа» (`ERROR: simulation timeout`).

Нормальные предупреждения, на них не обращайте внимания: `vopt-10908` (из-за `+acc` для осциллограмм) и `vlog-13314` в `motion_ctrl` (порт-структура `cmd`).

На что обратить внимание в каждом тесте

Тест	Главное, что проверяет
`tb_uart_rx`	288 байт, все 256 значений, скорость ±3 %, помеха 1/4 бита, ошибка кадра и восстановление, строка команды
`tb_byte_fifo`	порядок байт против эталонной очереди, заполнение (17 байт при DEPTH = 16), `overflow`, 5000 случайных тактов
`tb_cmd_parser`	45 строк: правильные, пустые, каждая ошибка err 1–4, восстановление после ошибки, долгое ожидание `cmd_ready`
`tb_udiv`	2013 делений против `/` и `%`, каждое ровно за 51 такт
`tb_motion_ctrl`	24 команды; на **каждый** импульс — ширина, период, пауза после DIR; число шагов по формуле Брезенхема в каждый момент; хоминг Z → X → Y; err 3, 5, 6
`tb_debounce`	дребезг не проходит, одно чистое переключение через ~50 тактов
`tb_reply_gen`	строки `ok`, `err 1` … `err 6` на линии; два результата подряд
`tb_uart_tx`	ваш тест передатчика
`tb_robot_top`	вся цепочка: строки по UART на 115200 → импульсы → ответы в «Termite»; две строки подряд без ожидания

Тесты проверены на «подложенных» ошибках: в копиях модулей я нарочно портил 10 мест (импульс на такт короче, нет паузы после DIR, Брезенхем стартует с 0, нет проверки концевика, нет проверки повтора поля, неверный масштаб дроби, чтение у края бита, FIFO не сбрасывает `rd_valid`, ответ без LF, антидребезг не обнуляет счётчик). Каждую из 10 ошибок тесты поймали.

---

6. Как перенести в Quartus

1. Создайте проект в отдельной папке (например `C:\FPGA_PROJECT\PasteRobot\quartus`), устройство **EP4CE6E22C8**, верхний модуль **robot_top**.
2. Добавьте файлы (Project → Add/Remove Files). Пакет — **первым**: `common/robot_pkg.sv`, `01_uart_rx/uart_rx.sv`, `02_byte_fifo/byte_fifo.sv`, `03_cmd_parser/cmd_parser.sv`, `04_udiv/udiv.sv`, `05_motion_ctrl/motion_ctrl.sv`, `06_debounce/debounce.sv`, `07_reply_gen/reply_gen.sv`, `08_uart_tx/uart_tx.sv`, `09_robot_top/robot_top.sv`. Тесты `tb_*.sv` в Quartus **не** добавлять.

3. SDC-файл с тактом: `create_clock -name clk_50 -period 20.000 [get_ports clk_50]`.
4. Назначьте выводы (Pin Planner) по схеме платы: `clk_50`, `key_rst_n`, `uart_rxd`, `uart_txd`, светодиоды и свободные выводы гребёнки для `motor_pul[0..3]`, `motor_dir[0..3]`, `motor_ena[0..3]`, `limit_x/y/z_raw`.

Проверено: проект собирается без ошибок, занимает **2917 из 6272 ячеек (47 %)**, 2048 бит памяти, 11 из 30 умножителей, и укладывается в 50 МГц с запасом 1,6 нс в худшем режиме (85 °C). Критичных предупреждений нет, кроме «выводы не назначены» (их назначите вы) и «`motor_ena` постоянно 0» (так и задумано, см. ниже).

---

7. Что проверить и решить перед подключением к роботу

- **Уровни 3,3 В → 5 В.** Входы TB6600 рассчитаны на 5 В. Нужен буфер (вопрос 8 в ТОП).
- **ENA.** Сейчас драйверы включены всегда (`motor_ena = ENA_LEVEL`). Какой уровень включает TB6600 — зависит от схемы подключения, проверьте по паспорту и поставьте параметр `ENA_LEVEL`.
- **Направления.** Если мотор крутится не туда — бит в `DIR_INVERT`. Если хоминг едет от концевика — бит в `HOME_DIR`. Особенно проверьте Z: в каком направлении «вверх» и где стоит его концевик.
- **Концевики.** `LIMIT_ACTIVE_LOW = 1` — нажатый концевик замыкает вывод на землю. Нужна подтяжка к 3,3 В.
- **Скорость.** `STEP_HZ = 2000` и `HOME_STEP_HZ = 1000` — начальные значения. Подберите на роботе частоту, на которой моторы не пропускают шаги.
- **Коэффициенты.** Подставьте настоящие k командой `M92`, когда будут известны микрошаг, ход винтов и диаметр шприца.

Что можно добавить следующим шагом

- файл `.qsf` с выводами под вашу плату и проводку;
- ответ на переполнение очереди (`err 7`) вместо одного светодиода;
- `M114` — вывод координат на ПК (координаты уже есть: выходы `pos_x/y/z` у `motion_ctrl`);
- кнопка аварийной остановки (E-Stop) через `motor_ena`;
- плавный разгон и торможение.